

Spine Formal-v6 Primary Dynamic-Graph Results

Correctness-Gated Spine vs. GraSU+ReGraph K4-shared

Chuxiao Han

Live snapshot, July 2026

Abstract

This live report contains 24 of 36 expected formal-v6 executions and 8 complete Spine/K4-shared pairs. Missing bars are still-running or queued executions, never zero-valued measurements. Every admitted row passes architecture-precision and independent mathematical oracles, full final-state comparison, and request, response, and DRAM conservation.

1 Measurement contract

Algorithm	System	Window	Warm state
CC	G+R K4-shared	complete_execution	False
CC	Spine	complete_execution	False
ResPR	G+R K4-shared	complete_execution	False
ResPR	Spine	complete_execution	False
SSSP	G+R K4-shared	complete_execution	False
SSSP	Spine	dynamic_update_only	True

Table 1: Measurement windows observed in admitted formal-v6 rows.

Dataset scope. The frozen corpus contains 11 real datasets; 9 enter the full-graph comparison. The remaining datasets, BC (24,575,382 vertices), UK (18,520,486 vertices), exceed the frozen Spine capacity of 16,777,216 vertices. They are capacity exclusions, not failed or selectively removed performance rows, and no slices replace them in the full-graph aggregate.

Synthetic endpoint. R19-32 (524,288 vertices and 15,483,485 unique directed edges) is reported as a separate scalability endpoint. It is never included in a real-dataset aggregate; the figures and cycle table place it after a dotted or ruled separator.

SSSP interpretation. Spine starts from a verified persisted old-graph SSSP state and measures the accepted update, automatic active-source discovery, incremental propagation, and drain. The conversion-free GraSU+ReGraph baseline performs its declared complete ReGraph execution because it has no equivalent persisted incremental SSSP state. This is an architecture-level dynamic-service comparison, not an identical-kernel microbenchmark.

Claim boundary. The report is partial until all expected rows finish. Device cycles, accepted memory bytes, and DRAMSim3 HBM energy are simulator outputs. They do not claim cycle-for-cycle FPGA calibration, on-chip dynamic energy, or total board power.

2 Current primary comparison

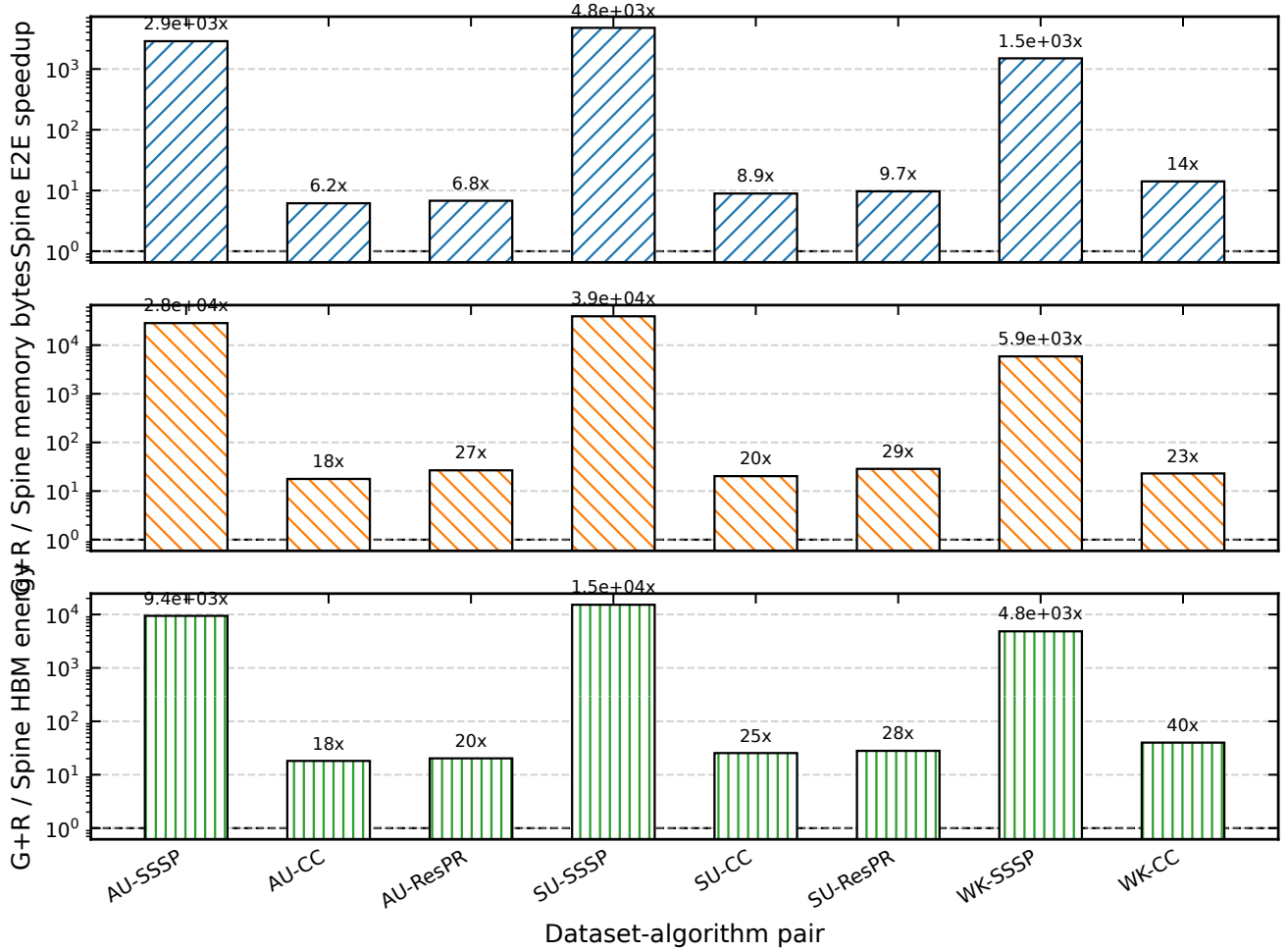


Figure 1: Ratios for complete insertion-batch-8 pairs. Values above one favor Spine for E2E latency and indicate that GraSU+ReGraph uses more memory traffic or HBM energy in the other panels. R19, when complete, appears to the right of the dotted separator and is not part of the real-dataset population.

Data	Algorithm	Spine cyc	G+R K4 cyc	Speedup
AU	SSSP	62,331	179,053,544	2872.62×
AU	CC	2,183,489	13,462,032	6.17×
AU	ResPR	2,172,364	14,729,898	6.78×
SU	SSSP	61,256	291,909,003	4765.39×
SU	CC	2,397,752	21,367,179	8.91×
SU	ResPR	2,390,535	23,162,486	9.69×
WK	SSSP	43,227	64,837,217	1499.92×
WK	CC	9,587,612	135,366,890	14.12×

Table 2: Absolute device cycles. Only cross-system final-state-matched pairs enter this table. R19 is separated from the real datasets by a rule.

3 Memory traffic and request locality

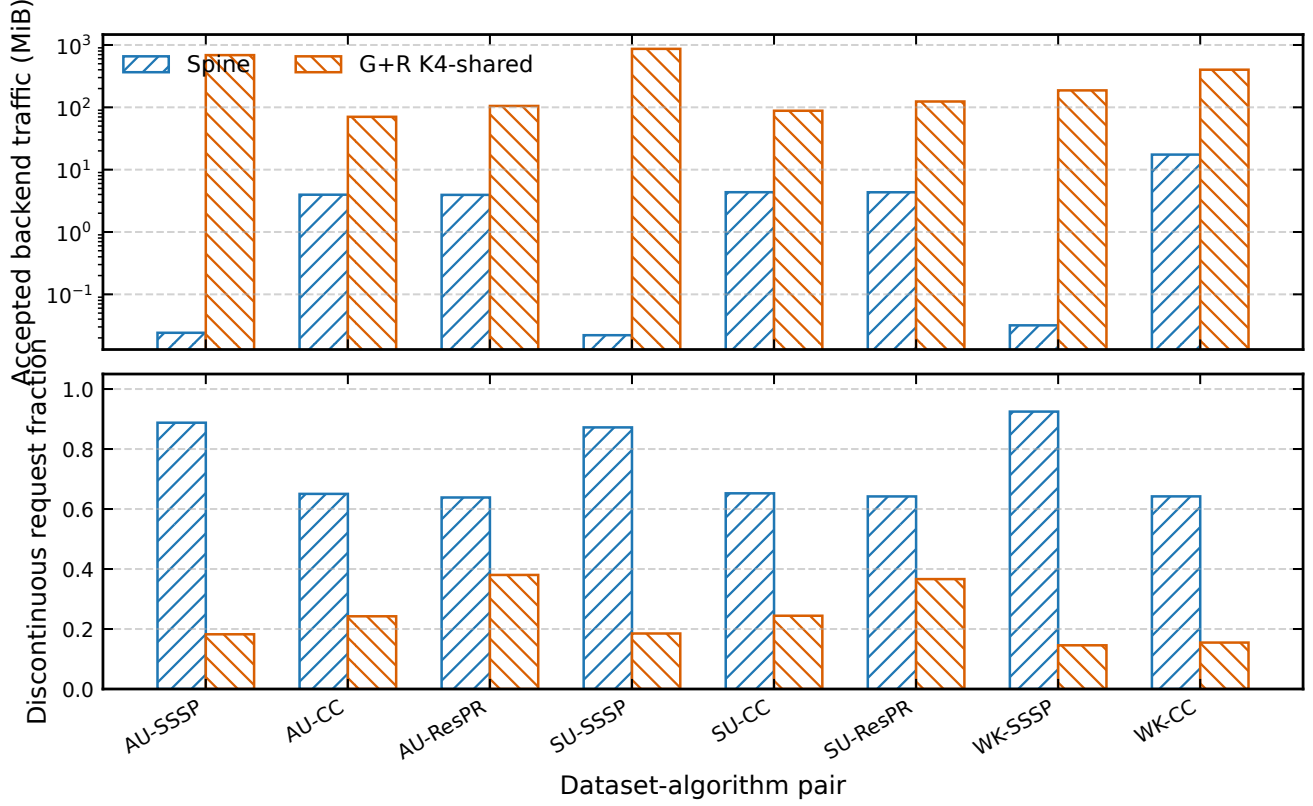


Figure 2: Absolute accepted-backend traffic and request-stream locality for the same complete pairs. A request is discontinuous when its accepted address does not continue the previous request from the same initiator, operation, and logical HBM channel. This is not a DRAM row-buffer-miss metric. R19 is a separate synthetic endpoint.

4 Update-only throughput

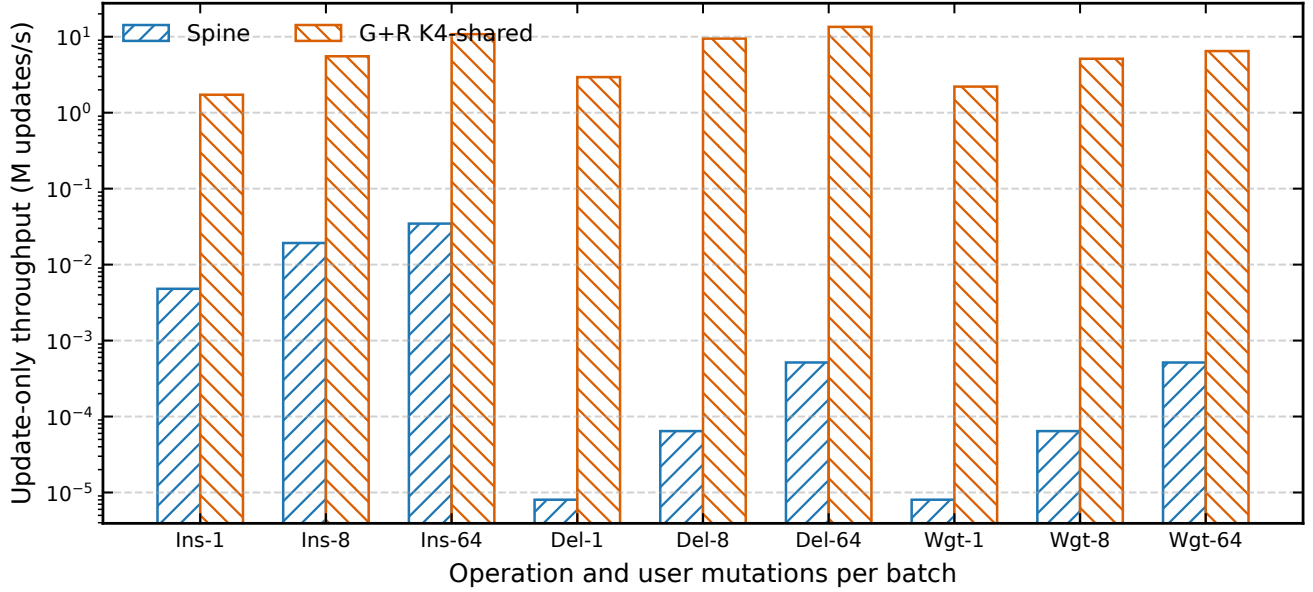


Figure 3: Correctness-gated AskUbuntu weighted-SSSP update-phase throughput. The current snapshot contains 9 of 9 planned operation–batch pairs. Ins uses the full materialized graph; Del and Wgt use the declared 64K-edge non-monotonic fallback slice. Missing points are not zeros.

Window boundary. This figure isolates graph-structure maintenance cycles and therefore exposes GraSU’s PMA update advantage. It must not be read as end-to-end dynamic graph service latency, which also includes differential discovery and propagation.

5 Simulator execution cost

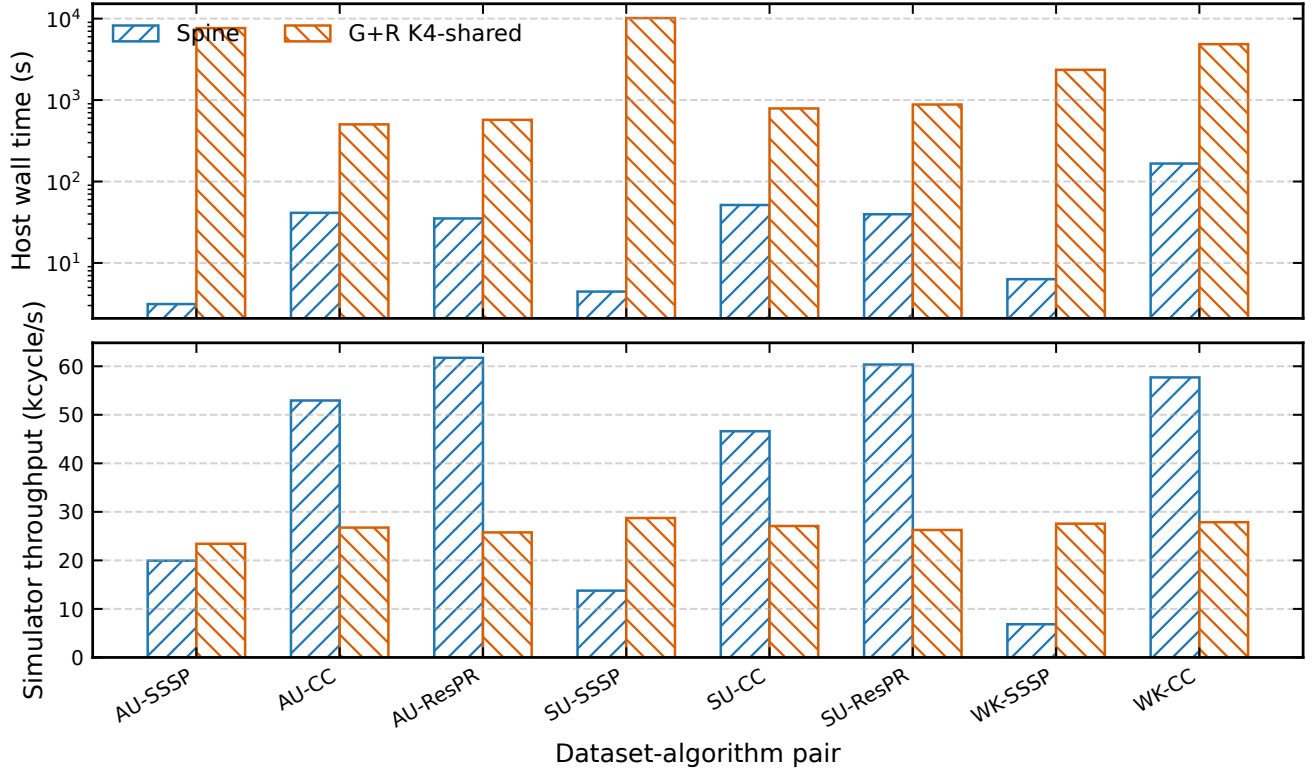


Figure 4: Host wall time and simulated-device-cycle throughput for the correctness-matched pairs. These are simulator engineering diagnostics under the recorded campaign concurrency, not accelerator latency or a cross-architecture performance metric.

Use boundary. Wall time establishes experiment feasibility and identifies simulator optimization targets. Architecture claims use device cycles and the shared memory model; they never use host wall time.

6 RQ3: realized work and latency

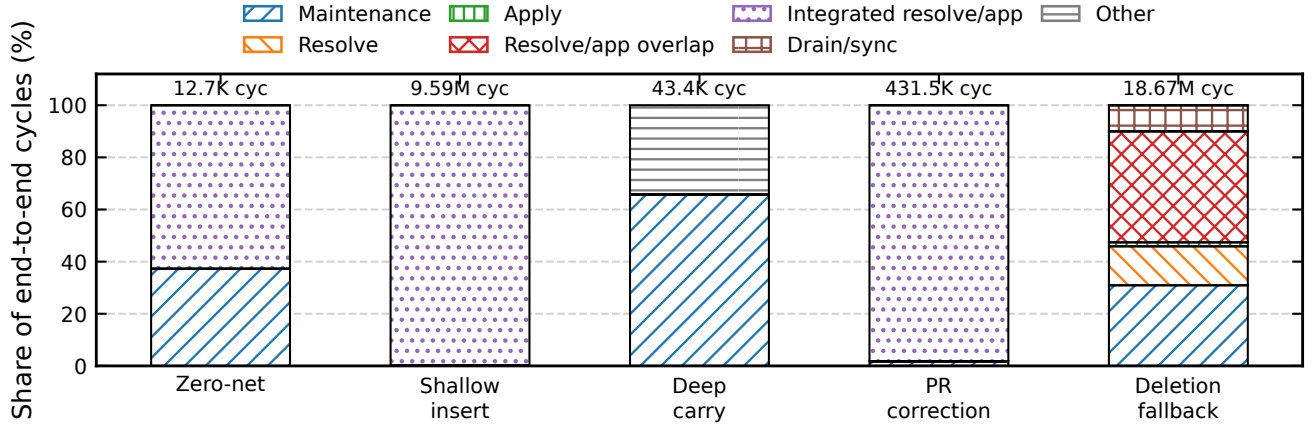


Figure 5: Exclusive, overlap-aware critical-path composition for zero-net, shallow insertion, deep carry, residual PageRank correction, and SSSP/CC nonmonotonic fallback. Absolute device cycles are printed above each bar.

Case	Algorithm	E2E cyc	Maint.	Integrated R/A	Sync
Zero-net	CC	12,671	4,720	7,951	0
Shallow insert	CC	9,587,612	7,390	9,580,222	0
Deep carry	SSSP	43,407	28,548	0	0
PR correction	Residual PR	431,479	7,525	423,954	0
Deletion fallback	SSSP	18,668,757	5,773,955	0	1,870,084

Table 3: Deterministically selected maximum-cycle representative of each eligible realized-work class.

7 RQ3: cost-model validation

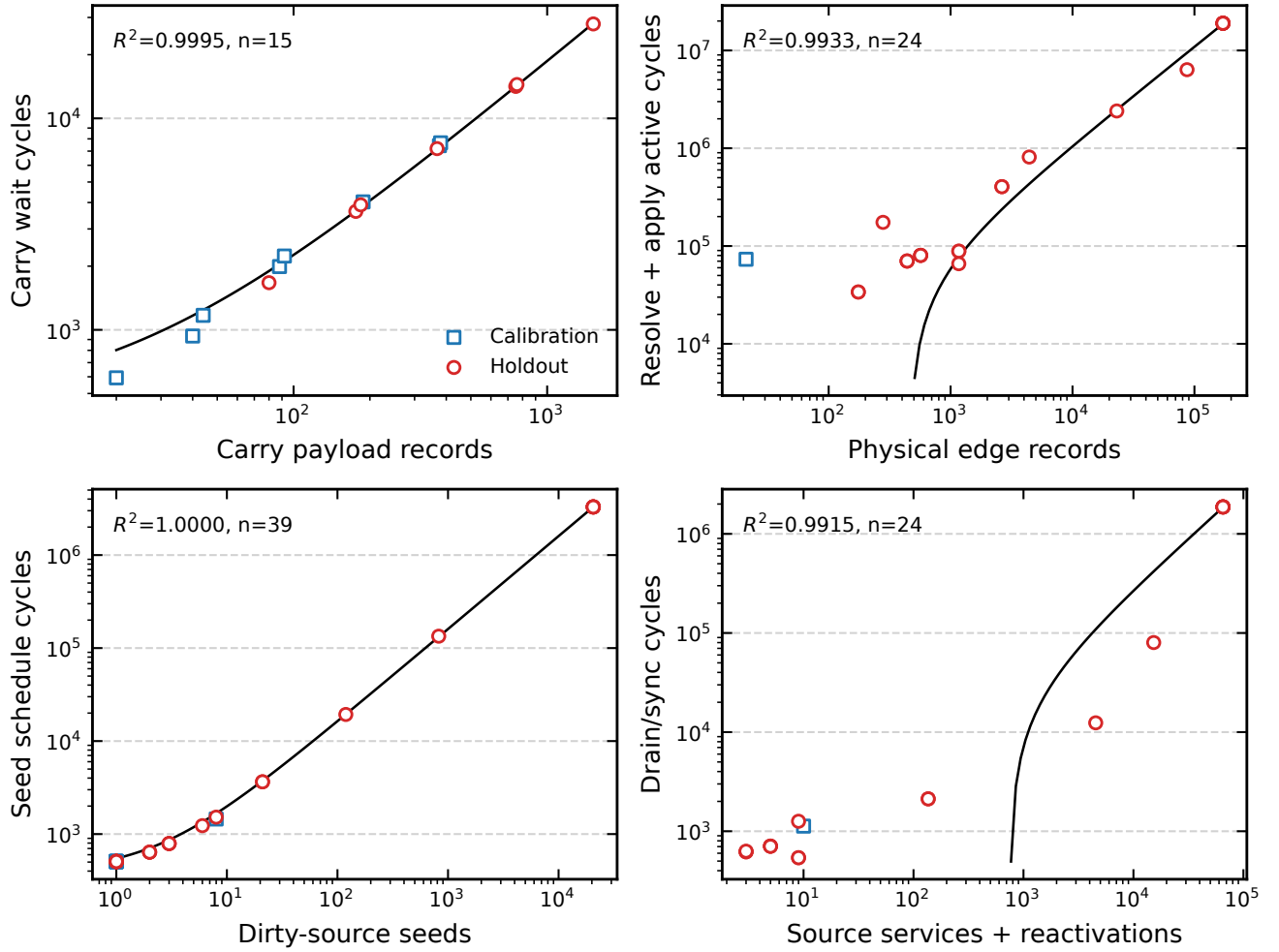


Figure 6: Direct execution counters versus mechanism-local cycles across 54 dual-oracle-admitted executions. Blue squares are calibration cases and red circles are holdout traces.

Mechanism	Samples	Slope	R^2
carry	15	18.242	0.9995
physical_resolve_apply	24	109.370	0.9933
seed	39	159.929	1.0000
drain_sync	24	28.773	0.9915

Table 4: Realized-work regressions. Carry work includes old payload reads, merge inputs, and output rewrites; cursor inspection is reported separately.

Interpretation. Carry, physical resolve/app, seed, and drain/synchronization work explain their mechanism-local cycle counters with $R^2 > 0.99$ in the covered population. Directory requests ($R^2 = 0.0011$) and aggregate switch work ($R^2 = 0.0181$) do not transfer across mixed classes, so the report makes no explanatory claim for those two counters.

RQ3 boundary. These correlations validate internal cost structure and bottleneck attribution; they are not cycle-for-cycle FPGA calibration or proof that one scalar predicts every topology.

8 Implementation-level power attribution

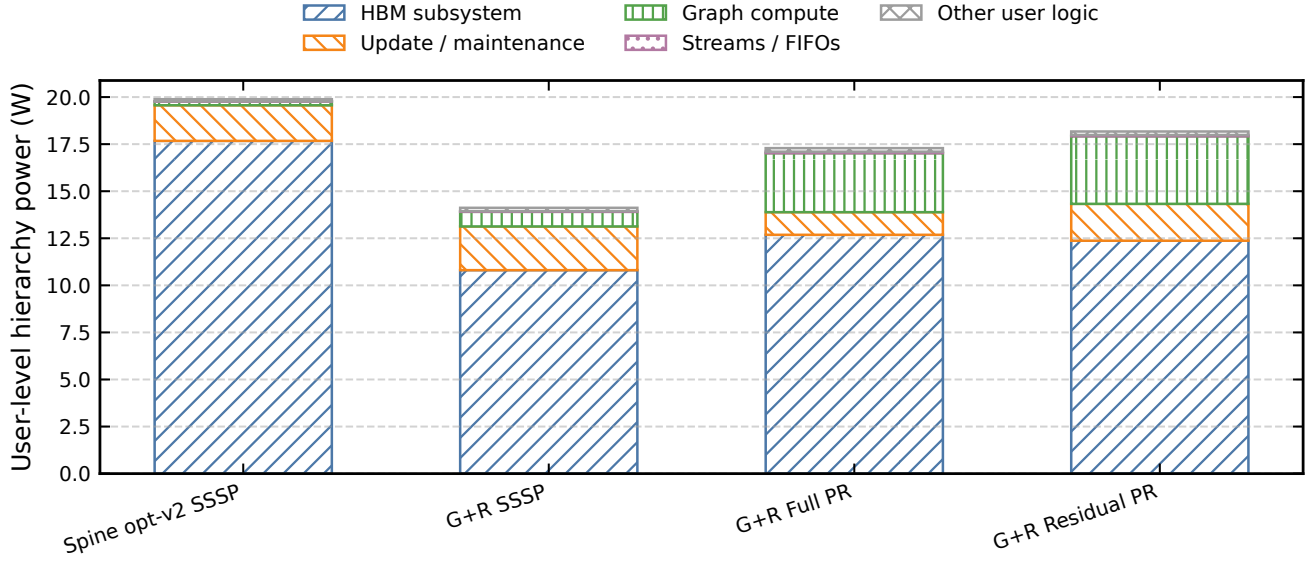


Figure 7: Vivado post-route vectorless hierarchy power at 150 MHz. The four bars are distinct routed builds; the Spine evidence covers SSSP only. Values use Vivado default activity with Low confidence and establish component attribution, not workload-calibrated energy or board power.

Energy boundary. The workload-specific HBM energy ratios in Figure 1 come from DRAMSim3 and are paired with the exact executions plotted there. The vectorless powers above must not be multiplied by the v6 latency to claim total workload energy. Workload-calibrated on-chip dynamic energy remains open because equivalent per-event energy models and complete activity counters are not yet available for every routed algorithm build.